

Fondamenti di TypeScript

Che Cos'è TypeScript?

TypeScript (TS) è un linguaggio di programmazione strettamente correlato a [JavaScript](#), ed è progettato per rendere il codice più **potente, affidabile e facile da gestire**. Analizziamolo in termini semplici:

Che cos'è TypeScript?

Immagina di costruire qualcosa con i mattoncini LEGO. JavaScript è come costruire con i LEGO, ma a volte **non puoi essere completamente sicuro** che i pezzi si incastrino perfettamente finché non provi effettivamente a montarli. Questo può causare problemi imprevisti durante l'esecuzione del programma.

Ora, **TypeScript** interviene come un manuale di istruzioni LEGO da supereroe. È come avere un **piano chiaro e delle etichette** per ogni pezzo LEGO. Con TypeScript, specifichi esattamente con quali tipi di pezzi stai lavorando, come dovrebbero collegarsi e cosa dovrebbero fare. Questo rende il codice meno soggetto a errori e più facile da comprendere.

Nota

La sintassi di TypeScript è quasi identica a quella di JavaScript. TypeScript presenta alcune differenze che migliorano e modernizzano il codice JavaScript.

Perché dovresti imparare TypeScript?

- **Meno errori:** TypeScript aiuta a individuare gli errori nel codice prima ancora di eseguirlo. È come avere un correttore ortografico per il codice;
- **Collaborazione migliorata:** quando si lavora con altri su progetti di grandi dimensioni, TypeScript aiuta tutti a comprendere meglio il codice, come se ci fossero etichette chiare su tutti i pezzi LEGO;
- **Produttività aumentata:** gli strumenti e le funzionalità di TypeScript possono rendere il programmatore più efficiente. È come avere strumenti speciali per costruire più velocemente;
- **Compatibilità con JavaScript:** TypeScript è costruito sopra JavaScript, quindi funziona perfettamente con tutto il codice e le librerie JavaScript esistenti.

In sintesi, TypeScript è come una **guida amichevole** che rende il percorso di programmazione più fluido. Aggiunge **sicurezza e chiarezza** al codice, offrendo un **grande vantaggio**, soprattutto quando si affrontano progetti più grandi e complessi. Quindi, se sei interessato allo sviluppo web o alla programmazione in generale, TypeScript è una competenza eccellente da acquisire!

Sintassi

TypeScript è un'estensione di JavaScript e viene applicato sopra questo codice. La sintassi di base di JavaScript e TypeScript è identica, quindi se già conosci le basi di JavaScript, troverai molto più semplice comprendere questo corso. Tuttavia, anche in questo caso, si consiglia di completare gli esercizi per ripassare i fondamentali.

Se non hai mai lavorato con JavaScript prima d'ora - nessun problema. In questo corso, verrà trattata la sintassi di base, che è la stessa sia per TypeScript che per JavaScript.

Iniziamo con la stampa di un testo sulla console:

```
console.log("Hello World");
```

Per visualizzare informazioni nella console, utilizziamo la seguente sintassi:

```
console.log("Text we want to output");
```

Nota che il testo che vogliamo stampare sulla console è racchiuso tra **doppi apici** (""). Puoi modificare il testo nella finestra del codice qui sopra per fare delle prove!

Nota

L'uso del punto e virgola (;) alla fine di una riga di codice è facoltativo.

Con questo comando, è possibile anche registrare qualsiasi cosa sulla console. Vediamo un esempio:

```
console.log("Any custom text you like")  
console.log(12345)
```

Nota

Non si usano le virgolette doppie (`""`) quando si stampano numeri; le virgolette doppie sono utilizzate solo per il testo!

È anche possibile combinare numeri e testo utilizzando i **template literal**. Questa sintassi è disponibile sia in TypeScript che in JavaScript.

Si devono usare gli backtick invece delle virgolette doppie. Inoltre, è necessario racchiudere i valori (numeri o nomi di variabili) in questa sintassi: `${variable/number}`. Vediamo un esempio:

```
var variable = 15
console.log(`There is two numbers: variableand{variable} and
variableand{13}`)
```

Nota

`var` è un modo ormai superato di dichiarare una variabile. Oggi, `let` è più comunemente utilizzato per la dichiarazione di variabili, mentre `const` viene usato per dichiarare costanti (variabili che **non possono essere modificate dopo l'inizializzazione**).

Nell'esempio di codice sopra, è possibile vedere come **abbiamo creato e utilizzato una variabile**. Nel prossimo capitolo, analizzeremo più da vicino come creare e utilizzare le variabili.

Dichiarazione Semplice di Variabile



Study more

Una variabile è un contenitore che memorizza dei dati. Puoi scrivere e modificare i dati in una variabile e utilizzare quella variabile ovunque sia necessario.

Le **variabili** semplificano notevolmente il lavoro di tutti gli sviluppatori. In una variabile è possibile memorizzare **testo**, **numeri** o un **valore booleano** (`true` o `false`). Vediamo la sintassi per creare una variabile e visualizzarla nella console:

```
let text = "Hello World!"
let number = 12345
let boolean = true

console.log(`First variable: text,secondvariable:{text}, second
variable: text,secondvariable:{number}, third: ${boolean}`)
```

Come puoi vedere, la **sintassi** per dichiarare una variabile è la seguente:

```
let variable_name = value
```

Inoltre, nota che quando utilizziamo la parola chiave `let` per dichiarare una variabile, sia TypeScript che JavaScript **determinano automaticamente** il tipo che questa variabile dovrebbe avere. TypeScript consente di **specificare il tipo di dato** di una variabile (da qui il nome TypeScript), di cui parlerò **più avanti in questo corso**. Per ora, concentriamoci sulle dichiarazioni semplici di variabili.

Nota

Ricorda che utilizziamo `let` e `const` invece di `var`. Per il momento, gli esempi includeranno ancora `var` perché rappresenta il modo più vecchio di dichiarare variabili in TypeScript e JavaScript, in uso **da molto tempo**. Successivamente, esamineremo i **metodi più recenti** per dichiarare variabili.

Operazioni aritmetiche in TypeScript

TypeScript, come qualsiasi altro linguaggio di programmazione, supporta **operazioni matematiche** semplici e complesse. Ecco una panoramica delle operazioni di base:

- **Addizione** (`+`): Somma di due numeri:

```
let sum = 5 + 3;  
console.log(sum)
```

- **Sottrazione** (`-`): Sottrae un numero da un altro:

```
let difference = 10 - 4;  
console.log(difference)
```

- **Moltiplicazione** (`*`): Moltiplica due numeri:

```
let product = 6 * 7;  
console.log(product)
```

- **Divisione** (`/`): Divide un numero per un altro:

```
let quotient = 20 / 4;  
console.log(quotient)
```

- **Modulo** (`%`): Restituisce il resto della divisione:

```
let remainder = 15 % 4;  
console.log(remainder)
```

- **Esponenziazione** (`**` o `Math.pow()`): Eleva un numero a una potenza:

```
let power = 2 ** 3;  
console.log(power)
```

Spero di aver elencato tutto. Questo è un elenco delle operazioni aritmetiche più comunemente utilizzate in TypeScript. Spiegherò come utilizzare queste operazioni nel prossimo capitolo, dove imparerai a usarle correttamente e cosa puoi ottenere grazie al loro utilizzo.

Matematica in TypeScript

Hai mai sentito dire che non serve la matematica per programmare? Mi dispiace deluderti, ma in realtà serve. Tuttavia, si tratta **solo delle basi**! In questo capitolo, esploreremo come applicare le tue conoscenze aritmetiche nella programmazione con TypeScript.

Iniziamo da ciò che già conosci. **Possiamo eseguire operazioni sui numeri** utilizzando gli strumenti discussi nel capitolo precedente. Vediamo alcuni esempi di codice:

```
console.log(150 + 150);  
console.log(900 / 3);
```

Questo è l'esempio più semplice di utilizzo delle operazioni matematiche in TypeScript. Tuttavia, potresti averlo già visto nel capitolo precedente, quindi vediamo un esempio più complesso in cui utilizziamo **operazioni multiple**:

```
let res = 20 * 10 - 75 / (22 + 3) - 2 ** 4;  
console.log(res);
```

È importante comprendere l'ordine di esecuzione delle operazioni matematiche. Dai tempi della scuola, potresti ricordare che le operazioni tra parentesi vengono eseguite per prime, seguite dall'elevamento a potenza e così via. Analizziamo l'espressione sopra per rinfrescare questi concetti:

Ogni espressione matematica può essere suddivisa in una sequenza di sottoattività. Dal video sopra, è evidente che **le operazioni tra parentesi** vengono eseguite per prime, seguite da **elevamento a potenza, moltiplicazione/divisione** e solo successivamente **addizione** e **sottrazione**. Semplice matematica di base.

Interazione tra numeri e variabili

Spero che l'ordine di esecuzione delle operazioni matematiche sia ora chiaro. Vediamo ora come possiamo **combinare variabili e numeri**:

```
let number_1 = 10;  
let number_2 = 15;  
console.log(number_1 + number_2);
```

Possiamo eseguire operazioni matematiche su due **variabili di tipo numerico**. Tuttavia, se una delle variabili ha un tipo diverso, l'operazione restituirà un risultato inatteso:

```
let num : number = 20;  
let str : string = '23';  
console.log(num + str);
```

Come si può osservare nell'esempio sopra, l'operazione matematica **non è stata eseguita**. Al contrario, è stata effettuata una **concatenazione**. Questo termine descrive l'aggiunta di stringhe tra loro. Tuttavia, non affrettiamoci a trarre conclusioni; proviamo a eseguire un'altra operazione matematica con le stesse variabili:

```
let num: any = 20;  
let str: any = '10';  
console.log(num - str);  
console.log(num / str);  
console.log(num ** str)
```

Sì, è possibile utilizzare operazioni matematiche (*eccetto l'addizione*) su diversi tipi di dati. Sì, è per questo che tutti sono entusiasti di JavaScript e TypeScript. No, non posso spiegare perché ciò accade. È necessario accettarlo come un dato di fatto.

Nota

Il compilatore TypeScript **produrrà errori**, ma considererà comunque tali espressioni. Questo accade perché **TypeScript viene traspilato in JavaScript** dopo l'esecuzione del codice.

È possibile utilizzare operazioni matematiche tra una variabile e un numero?

Sì.

```
let num = 30;  
console.log(num - 10);
```

Nota

A differenza di JavaScript, il compilatore TypeScript evidenzia un errore quando si tenta di sottrarre una stringa da un numero. Questo codice verrà eseguito, ma riceveremo un avviso che stiamo commettendo un errore.

Dichiarazione di Variabile Tipizzata

Poiché TypeScript è un superset di JavaScript, non ci sono molte differenze nella sintassi di base. Tuttavia, esaminiamo una delle differenze più importanti: la **tipizzazione**.

TypeScript introduce il concetto di **variabili tipizzate**, in cui il tipo di ogni variabile è definito esplicitamente. Tra i vari tipi di dati supportati da TypeScript, tre dei più utilizzati sono `number`, `boolean` e `string`.

In precedenza, dichiaravamo le variabili utilizzando la parola chiave `var` e TypeScript **inferiva automaticamente il tipo** di quella variabile. Ora, semplificheremo il compito di TypeScript e **definiremo manualmente il tipo della variabile**. Questo viene fatto nel seguente modo:

```
let one : number = 10;  
console.log(one)
```

La sintassi è leggermente diversa e possiamo vedere quale tipo avrà la nostra variabile `one`. Esaminiamo più da vicino la sintassi:

```
let variable_name : type = value;
```

Allo stesso modo, possiamo definire variabili degli altri due tipi, ad esempio una stringa:

```
let variable : string = 'Hello c<>definity!';  
console.log(variable);
```

Avrai notato che sembra quasi di impartire un comando al nostro codice: **La variabile sia una stringa!**

In questo modo, sarà più semplice ricordare la sintassi.

boolean

Passiamo ora al tipo `boolean`, che è un tipo di dato che può contenere solo i valori `true` e `false`.

D: A cosa serve?

R: Serve per eseguire blocchi di codice con condizioni.

D: Cosa sono i blocchi di codice con condizioni?

R: Li approfondirai più avanti in questo corso.

Vediamo una variabile di tipo boolean:

```
let variable : boolean = true;  
console.log(variable);
```

A cosa serve la tipizzazione dei dati?

Principalmente, la tipizzazione dei dati aiuta un programmatore a **comprendere meglio il proprio codice**. Si ha la libertà di decidere se utilizzare o meno la tipizzazione, ma TypeScript è molto apprezzato per questa caratteristica.

Inoltre, **la tipizzazione dei dati aiuta a evitare errori**, poiché il compilatore TypeScript **evidenzierà i frammenti di codice** in cui i tipi di dati non corrispondono, come mostrato nello screenshot:

```
1 let one : number = 10;  
2 let two : string = '5';  
3 console.log(one / two);
```

Abbiamo definito variabili con i tipi `number` e `string`. Poi proviamo a dividere un valore numerico per una stringa. Il compilatore ci avverte che **potremmo commettere un errore**. Il compilatore si prende cura di noi.

Commenti

D: Cosa fa un programmatore quando deve scrivere la documentazione per il proprio codice?

R: La scrive in un **file Word**.

D: Cosa fa un buon programmatore quando deve scrivere la documentazione per il proprio codice?

R: Usa i **commenti**.

Trasformiamoci in un buon programmatore e impariamo anche come utilizzare i commenti all'interno del codice. Ma prima, scopriamo cosa sono:



Study more

Commenti nel codice sono note testuali o annotazioni aggiunte dagli sviluppatori al codice sorgente di un programma. **Non vengono eseguiti dal computer e sono ignorati durante l'esecuzione del programma.** I commenti vengono utilizzati per spiegare il codice, rendendolo più comprensibile ad altri sviluppatori o per consultazioni future.

In TypeScript, esistono due tipi di commenti: 1. Possiamo commentare un'intera riga usando `//`; 2. Possiamo anche commentare un blocco di codice che si estende su più righe o un frammento di codice su una sola riga usando `/* code */`.

Vediamo un esempio in cui dobbiamo commentare alcune righe e scrivere dei commenti. Ripeto, **il compilatore non eseguirà né prenderà in considerazione i frammenti di codice commentati**, quindi possiamo scrivere qualsiasi cosa lì (*tranne le parolacce; mia madre mi rimprovera se le scrivo*).

```
console.log("Hello world!")
// This code performs output of information to the screen.
let word : string = 'c<>definity'
console.log(`Hello ${word}!`)
/*This code creates a variable of type string
and uses it for output to the screen.
We use this variable with the required syntax
and successfully display text on the screen with the variable. */
```

Come puoi vedere nel codice sopra, abbiamo commentato vari frammenti di testo. Nei commenti, scriviamo spiegazioni su ciò che accade nel codice e su cosa fanno determinate righe. **Nota che quando si commentano più righe, si utilizza una sintassi di commento diversa.** (`/* */`) Dopo `/**`, tutto ciò che si trova a destra dei caratteri viene commentato.

Possiamo anche commentare frammenti di codice. Ad esempio, se si sospetta un **errore** in una determinata riga, è possibile commentare quella riga ed eseguire nuovamente il codice per verificare la presenza di errori in quella riga.

```
//var str = 'Hello World';
var num = 132;
console.log(num/* - str*/);
```

Sopra, abbiamo commentato frammenti di codice in cui sospettiamo errori utilizzando **diversi tipi di commenti**. Personalmente, identifico spesso le aree soggette a errori nel codice usando questo metodo e **consiglio** di fare lo stesso. È una competenza utile per un buon programmatore.

Inoltre, non dimenticare di **scrivere la documentazione** per il tuo codice. È come mettere una bottiglia d'acqua accanto al letto dopo una notte di festa.

Sfida: Correzione della Sintassi del Codice

Questo codice contiene alcuni **errori di sintassi**. Il compito consiste nell'identificare le posizioni degli errori e commentarle per consentire l'esecuzione corretta del codice.

```
let one : number = 5;  
let two : string = one ** 4;  
let result = two - one;  
console.log('code has been executed succesfully')  
console.log(`the result is ${result}`)
```



Suggerimento



Soluzione